

LeNet-5: MNIST, Compression Robustness Analysis

*A Controlled Study of Compression versus Robustness
in LeNet-5 on MNIST and MNIST-C*



Project Report

CS 725 – Foundations of Machine Learning

Guide: Prof. Abir De

Full Name	Roll Number
Harshith Matta	25M0834

Department of Computer Science & Engineering
Indian Institute of Technology Bombay
November, 2025

Abstract

Iterative magnitude pruning can remove roughly 96% of LeNet-5’s weights with no loss of clean MNIST accuracy. That result is almost always reported on a single axis, clean test accuracy, and read as evidence that the compression was free. A deployed model, however, meets blurred digits, shifted digits, sensor noise, and occasionally adversarial perturbation. This project asks whether compression that is invisible on clean MNIST stays invisible under corruption and attack.

The answer has two parts, and the second needed a control experiment to find. Under the textbook layer-wise recipe, compression is *not* free: at 3.75% of weights remaining, pruning costs a standard-trained network 0.56 points of clean accuracy but 3.89 points of MNIST-C accuracy, a sevenfold hidden cost that clean-accuracy validation cannot see. Yet most of that gap turns out to be the pruning *schedule*, not the sparsity. Re-running with global magnitude pruning at matched sparsity and identical seeds, the corruption penalty disappears (0.13 points, $p = 0.83$), and global pruning beats layer-wise by 3.76 points head-to-head. The defensible conclusion is therefore narrower than the project set out to prove, and more useful: the standard pruning recipe costs corruption robustness that the sparsity itself does not require. All numbers carry an error bar over five seeds and are regenerated from 260 recorded runs.

Contents

Abstract	2
1 Introduction	4
2 Phase 1: Reproduction and Methodological Audit	4
2.1 Motivation	4
2.2 The three protocol rules	4
2.2.1 Test-set model selection	4
2.2.2 Normalisation lives inside the model	5
2.2.3 Every run writes a record	5
3 Phase 2: The Compression Pipeline	5
3.1 Setup	5
3.2 Headline results	6
3.2.1 Pruning reproduces the Lottery Ticket result, then diverges by metric . .	6
3.3 Distillation recovers part of the cost, for free	6
3.4 INT8 buys size and costs latency	6
3.5 Pruning increases adversarial robustness at small ϵ	7
3.6 The surviving first-layer filters are not edge detectors	7
4 Phase 3: Isolating the Cause with a Schedule Control	8
4.1 The confound, stated precisely	8
4.2 The control	8
4.3 The schedule effect is not uniform across training conditions	9
5 Statistical Methodology	10
6 Two Measurement Corrections That Changed the Answers	10
6.1 PGD-40 with one restart overstates robustness	10
6.2 Adversarial training on sparse networks needs an ϵ ramp	10

7	Limitations	11
8	Conclusion	11
	References	11

1 | Introduction

Neural-network compression is usually validated on the same distribution it is trained on. For a classifier this means clean test accuracy: prune the network, confirm accuracy has not dropped, and declare the removed weights unnecessary. This is the standard reading of the Lottery Ticket Hypothesis, and on clean MNIST it holds cleanly, LeNet-5 tolerates removal of about 96% of its weights with no measurable accuracy loss.

The reading has a blind spot. Clean accuracy measures behaviour on exactly one distribution, whereas a deployed model faces distribution shift, blur, translation, noise, and adversarial input. If compression quietly damages behaviour on those axes while leaving clean accuracy untouched, standard validation will call the compression free when it is not. This project investigates that gap directly, on a testbed small enough that every claim can be checked exhaustively: LeNet-5 on MNIST, with MNIST-C for corruption robustness and PGD for adversarial robustness.

The work proceeds in three connected phases. The first phase reproduces LeNet-5 faithfully and audits the reproduction itself, uncovering a test-set selection leak that motivated three methodological rules carried through the rest of the project. The second phase builds a full compression pipeline, iterative magnitude pruning, knowledge distillation, and INT8 quantization, and measures each stage against corruption and adversarial robustness rather than clean accuracy alone. The third phase isolates the cause of the corruption penalty with a controlled comparison of pruning *schedules*, and it is here that the headline finding emerges: the naive layer-wise recipe, not the sparsity, is responsible for most of the hidden cost.

Section 2 describes the reproduction audit and the resulting protocol. Section 3 covers the compression pipeline and its per-stage results. Section 4 presents the schedule control experiment. Section 5 reports the statistical methodology, and Section 6 the two measurement corrections that changed the answers. Sections 7 and 8 give limitations and conclusions.

2 | Phase 1: Reproduction and Methodological Audit

2.1 Motivation

Before measuring how compression affects robustness, the underlying LeNet-5 reproduction had to be trustworthy. The repository keeps Part 1, a faithful port of LeCun et al. (1998), in a frozen `legacy/` directory, and an audit of that port found one defect serious enough to invalidate its headline comparison. Three conventions were adopted for the rest of the project as a direct result.

2.2 The three protocol rules

2.2.1 Test-set model selection

Part 1 selected its reporting epoch by test accuracy. On this architecture that choice inflates the reported number by 0.05 to 0.08 points, small in absolute terms, but the same order of magnitude as the data-augmentation effect the number was being used to demonstrate. Part 2 instead selects the reporting epoch on a 5,000-image validation split held out of the training data, and touches the test set exactly once per configuration. Both protocols are reported side by side so the size of the leak is visible rather than asserted.

Once selection is done correctly, the augmentation gap from the original Part 1 slides falls inside seed variance. That is itself a finding about the reproduction, and it is why every number in this report carries a $\pm$ over five seeds.

Table 1: Clean accuracy under correct versus leaky model selection (five seeds).

Protocol	standard	augmented
Correct — best epoch on validation	99.08 $\pm$ 0.09	99.16 $\pm$ 0.04
Leaky — best epoch on test	99.13 $\pm$ 0.07	99.24 $\pm$ 0.03

2.2.2 Normalisation lives inside the model

Input normalisation is applied as a network module, so the model consumes raw $[0, 1]$ pixels. This is load-bearing, not cosmetic. If normalisation were a dataset transform instead, an $\varepsilon = 0.3$ attack measured in standardised space would really be $\varepsilon \approx 0.092$ in pixel space, and every adversarial number would be silently wrong by a factor of $1/0.3081$. A pixel-space assertion runs at the top of every path that attacks a model.

2.2.3 Every run writes a record

Every one of the 260 runs writes a JSON record carrying a config hash, seed, git SHA, machine spec, metrics, and wall-clock time. Every table and figure in this report is regenerated from those records by a script; none is hand-edited. A further 27 records of failed configurations are kept rather than deleted, so a reader can see what did not work.

3 | Phase 2: The Compression Pipeline

3.1 Setup

The model is LeNet-5 with dense C3 connections and tanh activations, 61,706 parameters. MNIST is split 55,000 / 5,000 / 10,000 with a fixed seed; MNIST-C (Mu & Gilmer, 2019) supplies 15 corruptions over 10,000 images each, reported both per-corruption and as the mean. Training is SGD at learning rate 0.01, momentum 0.9, batch 64, for 20 epochs, with an identical epoch budget for every condition. Compute was 44 recorded GPU-hours on NVIDIA A40 cards scheduled through SLURM; latency is measured on CPU, batch 1, single-threaded, as the median of 1000 iterations.

The pipeline has three compression stages. *Pruning* is iterative magnitude pruning, removing 20% of surviving weights per round over 20 rounds. *Distillation* is Hinton knowledge distillation, with temperature and α chosen on validation from a 3×3 grid and a double-width LeNet teacher. *Quantization* is eager-mode INT8 quantization-aware training via `torch.ao.quantization`, with real conversion and measured on-disk size and CPU latency.

```

1 def prune_step(model, scope, rate=0.20):
2     # rank weights by magnitude, then zero the smallest 'rate' fraction
3     if scope == "layerwise":
4         for layer in prunable_layers(model):
5             threshold = magnitude_quantile(layer.weight, rate)
6             layer.mask &= (layer.weight.abs() >= threshold)
7     elif scope == "global":
8         all_weights = concat_magnitudes(model)      # whole network at once
9         threshold = magnitude_quantile(all_weights, rate)
10        for layer in prunable_layers(model):
11            layer.mask &= (layer.weight.abs() >= threshold)
12    apply_masks(model)
13    return model

```

Listing 1: Iterative magnitude pruning loop, one round.

3.2 Headline results

Table 3 collects the headline rows of the full results table.

Table 2: Headline configurations. Clean and MNIST-C are accuracy (%); Robust is worst-case PGD accuracy at $\varepsilon = 0.3$; Size is measured on-disk KB.

Configuration	Clean	MNIST-C	Robust	Size (KB)	Lat. (ms)
Standard, dense	99.08 ± 0.09	73.72 ± 0.92	0.00	245.6	0.366
Standard, 3.75% weights	98.52 ± 0.14	69.83 ± 2.41	0.00	245.6	0.365
Standard, 1.30% weights	95.65 ± 0.79	64.03 ± 1.42	0.00	245.6	0.367
Adversarial, dense	98.78 ± 0.09	78.48 ± 0.79	90.24 ± 0.32	245.6	0.357
Adversarial, 3.75% weights	96.60 ± 0.79	69.99 ± 1.56	83.55 ± 1.75	245.6	0.356
IMP + KD + INT8	98.65 ± 0.17	71.10 ± 1.47	—	75.0	0.396
Global-pruned, 3.52%	98.90 ± 0.07	73.59 ± 0.98	0.00	245.6	—
Global-pruned, 1.15%	98.20 ± 0.16	70.42 ± 1.75	0.00	245.6	—

3.2.1 Pruning reproduces the Lottery Ticket result, then diverges by metric

Clean accuracy is flat, or slightly above the dense baseline, down to 13.8% of weights and degrades only below that, the sanity check the pruning implementation had to pass, and it passed. But MNIST-C accuracy falls faster at every level. At 3.75% weights the drop is 0.56 points clean against 3.89 points corrupted, a sevenfold difference. Reporting only the clean number would have called this compression nearly free.

Among three iterative-pruning variants, rewind-to-init and late rewinding are indistinguishable throughout, while fine-tuning without reset is 6.73 points worse at 1.30% weights. With a per-seed standard deviation of 5.89 points, however, that difference does not survive Welch’s t-test at five seeds ($p = 0.062$ raw, 0.174 Holm-adjusted). The Lottery Ticket ordering is directionally reproduced but statistically unestablished at this seed count.

3.3 Distillation recovers part of the cost, for free

Distilling into the pruned student lifts clean accuracy from 98.52% to 98.64% and MNIST-C from 69.83% to 71.49%, at identical parameter count, file size, and latency. Against standard training at the same budget, distillation alone is worth 0.17 points of clean accuracy ($p = 0.011$), which clears the roughly 0.09-point seed-noise floor. The temperature/ α grid, by contrast, found nothing, the entire 3×3 grid spans 0.16 points, below what one seed can resolve. The effect is from distilling at all, not from the particular temperature.

3.4 INT8 buys size and costs latency

INT8 quantization is $3.28\times$ smaller on disk (245.6 to 75.0 KB, measured rather than inferred from bit width) for 0.17 points of accuracy. It is also *slower*: batch-1 median latency rises from 0.359 to 0.383 ms, a significant $1.07\times$ slowdown ($p = 0.0005$). On a model this small the per-tensor quantize/dequantize overhead exceeds what integer arithmetic saves. Note also that unstructured pruning does not shrink the file at all, a pruned FP32 checkpoint is byte-for-byte the size of a dense one, because the zeros are still stored. Quoting a theoretical compressed size as “the model size” would misrepresent the result, so both the real and theoretical sizes appear in every record.

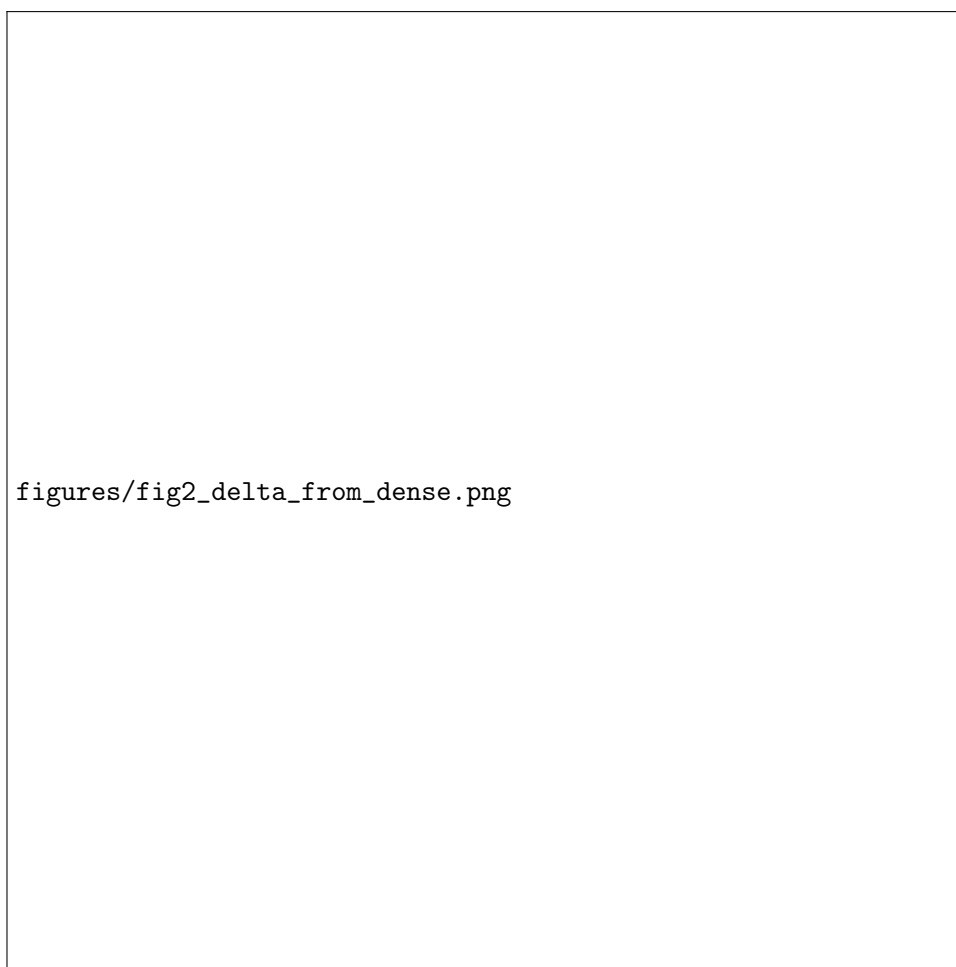


Figure 1: Cost-from-dense as weights are removed. The clean-accuracy curve stays near zero while the corruption curve pulls away, this gap is the central observation of the project.

3.5 Pruning increases adversarial robustness at small ε

The pruned model shows a 30-point gap at $\varepsilon = 0.05$ ($p = 0.0010$) and 12 points at $\varepsilon = 0.1$ ($p = 0.0046$), with non-overlapping per-seed distributions. Because this is the shape a broken attack produces, it was tested before being believed: black-box transfer from the dense model is weaker than white-box PGD on the pruned model (68.15% versus 14.10% at $\varepsilon = 0.1$), which rules out gradient masking. The effect is real but it is not a defence, both models sit in the 0 to 20% band and reach zero by $\varepsilon = 0.2$. It is better read as the other face of the first-layer damage discussed next.

3.6 The surviving first-layer filters are not edge detectors

LeNet-5’s original justification is a learned hierarchy of local feature detectors. At 3.75% of weights, the first layer retains 6 of 150 coefficients and half its filters are entirely empty; at 1.30%, two coefficients survive. What remains are isolated weights with no 5×5 structure left to be oriented. This is a confound rather than a curiosity: uniform per-layer pruning starves the 150-weight first layer long before the 48,000-weight later layers, so at the extreme end of every curve, *compressed* and *first layer destroyed* are the same condition. Separating them is exactly what the next phase does.

Table 3: Worst-case PGD accuracy across attack strength. Pruned models are more robust than the dense model at small ε .

Model	$\varepsilon = 0.05$	$\varepsilon = 0.1$	$\varepsilon = 0.15$	$\varepsilon = 0.3$
standard, dense	51.98 ± 9.51	2.33 ± 0.90	0.01	0.00
standard, 13.8% weights	82.18 ± 3.92	14.74 ± 5.02	0.13	0.00
adversarially trained, dense	98.09	97.29	96.19	90.55

Table 4: Survival of the first convolutional layer under layer-wise pruning.

Weights remaining	conv1 weights alive	filters entirely empty
100%	150/150	0/6
13.8%	21/150	0/6
3.75%	6/150	3/6
1.30%	2/150	4/6

4 | Phase 3: Isolating the Cause with a Schedule Control

4.1 The confound, stated precisely

The Phase 2 corruption penalty admits two explanations that the layer-wise schedule cannot separate: either *sparsity hurts corruption robustness*, or *the first layer was destroyed*. Uniform per-layer pruning applies the same proportional rate to the 150-weight first layer and the 48,000-weight later layers, so the first layer is reduced to a handful of coefficients while later layers stay comfortable. Any corruption penalty measured this way is a mixture of the two effects.

4.2 The control

Global magnitude pruning separates them. The protocol, rewind, seeds, and 20-epoch budget are held identical; the only change is that magnitudes are ranked across the whole network rather than within each layer. Global pruning distributes damage almost inversely, keeping the first layer at 26.7% of its weights at round 20 where layer-wise leaves it at 1.3%.

Table 5: Corruption penalty by pruning schedule at matched sparsity. The sevenfold penalty under layer-wise pruning disappears under global pruning.

Weights	Schedule	Δ clean	Δ MNIST-C	Ratio
3.75%	layerwise	-0.56	-3.89 ($p = 0.019$)	7.0×
3.52%	global	-0.18	-0.13 ($p = 0.83$)	0.7×
1.30%	layerwise	-3.43	-9.70	2.8×
1.15%	global	-0.88	-3.30	3.8×

At 3.5% weights the corruption penalty disappears entirely under global pruning, and head-to-head global is 3.76 points better on MNIST-C ($p = 0.021$) and 0.38 points better on clean accuracy ($p = 0.002$) at matched sparsity, strictly better on both axes. A residual divergence does survive near 1% weights, where global still loses 3.30 points of MNIST-C against 0.88 points of clean accuracy. So sparsity does eventually cost corruption robustness, but far later and far less than the layer-wise schedule implied.

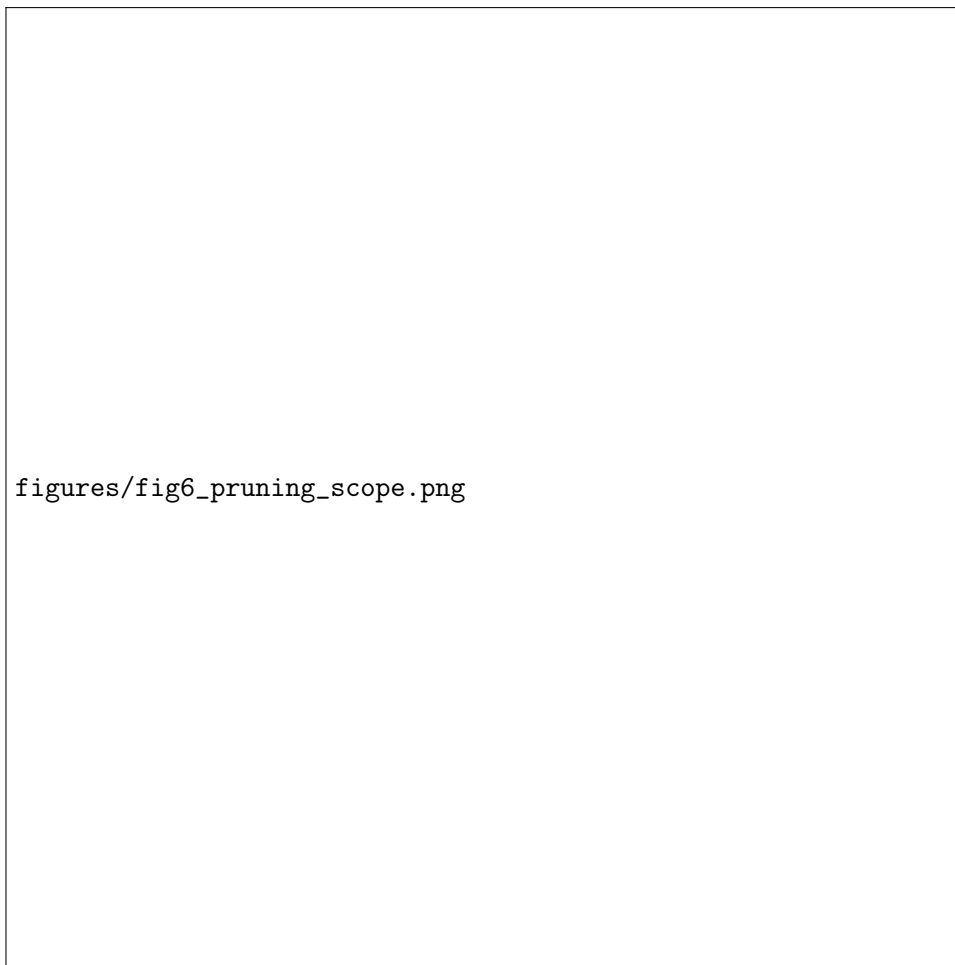


Figure 2: Layer-wise versus global pruning. At matched sparsity global pruning is strictly better on both clean and corruption accuracy until roughly 1% of weights remain.

4.3 The schedule effect is not uniform across training conditions

Repeating the control for the augmented and adversarial training conditions shows the schedule effect is condition-dependent, which is itself informative.

Table 6: Effect of switching from layer-wise to global pruning, by training condition, at roughly 13.5% weights.

Condition	Effect on MNIST-C of switching to global
standard	+ 2.15 points ($p = 0.010$)
augmented	+0.34 points ($p = 0.74$)
adversarial	+ 3.20 points ($p = 0.003$)

Augmentation already trains the network to tolerate the spatial perturbations a damaged first layer stops handling, so it has little left to gain; standard and adversarial training both do. The adversarial arm also becomes markedly more *stable* under global pruning, robust accuracy of 88.51 ± 1.17 against 84.72 ± 5.86 under layer-wise, so the layer-wise schedule was not only costing accuracy but making training unreliable.

The practical form of the finding: a deployment that prunes with the textbook layer-wise recipe and validates on clean MNIST ships a model measurably worse under corruption for no reason, because the global schedule reaches the same sparsity with both better clean accuracy

and better corruption accuracy.

5 | Statistical Methodology

Every quantitative claim rests on 59 comparisons across 13 families, each a Welch’s two-sided t-test at $n = 5$ seeds, with Holm-Bonferroni correction applied within each family at $\alpha = 0.05$. Of the 59 comparisons, 31 survive correction. Every comparison the report makes is listed in the accompanying significance table, *including the ones that fail*, since a results section that reports only its significant tests is selecting on outcome.

The comparisons that do not survive are worth naming: the Lottery-Ticket-versus-fine-tuning gap at 1.30% weights, the temperature/ α ablation, the INT8 accuracy drop, and adversarial-versus-standard corruption accuracy at every pruned level. With five seeds these tests have low power, so “not significant” means “not resolvable at $n = 5$ ”, not “no effect”. As a cross-check, every p-value was computed twice, once with `scipy.stats.ttest_ind` and once with a hand-written Welch implementation, and all 59 agree to within 10^{-6} .

6 | Two Measurement Corrections That Changed the Answers

Both corrections below produced plausible, publishable, wrong numbers before being caught. They appear here because the corrections are more instructive than the final values.

6.1 PGD-40 with one restart overstates robustness

Adversarially trained models were originally evaluated with the same attack they were trained against, PGD-40 with a single restart, and scored $93.40 \pm 0.24\%$. Under PGD-100 with 5 restarts the same checkpoints score $90.63 \pm 0.53\%$ ($p = 0.0001$), with the overstatement carrying the same sign on all five seeds. A step-and-restart sweep shows why: accuracy saturates by PGD-200 at one restart but keeps falling as restarts are added. Restarts, not steps, were the binding constraint. Four gradient-masking checks, monotonic decay in ε , PGD dominating FGSM, PGD-100 no weaker than PGD-40, and transfer weaker than white-box, confirm the underlying robustness is real; PGD-40 was simply an under-powered ruler.

6.2 Adversarial training on sparse networks needs an ε ramp

Training at the full $\varepsilon = 0.3$ from epoch 1, the recipe that works on the dense model, collapses every pruned ticket to constant prediction: loss pinned at $\ln(10)$ for all 20 epochs and all five seeds identical at 11.35%, the frequency of the most common class. Ramping ε over the first epochs fixes it outright.

Table 7: Robust accuracy under three adversarial-training recipes. The cold start at full ε collapses sparse networks; the ramp rescues them.

Recipe	21.3% weights	3.75% weights
SGD 1e-2, ε ramped	96.9 $\rightarrow$ 95.0	11.2 $\rightarrow$ 93.2
Adam 1e-4, ε ramped	84.5 $\rightarrow$ 74.3	55.2 $\rightarrow$ 54.0
SGD 1e-2, $\varepsilon = 0.3$ from epoch 1	11.2 (stuck)	11.2 (stuck)

A dense network survives a cold start at $\varepsilon = 0.3$; a sparse one does not, because there is no useful gradient when every example is already adversarial to a network that has learned nothing. The required ramp length is itself sparsity-dependent. Reporting the original collapse would have produced the false headline “pruned networks cannot be adversarially trained.”

7 | Limitations

- **The pipeline and ε -sweep grids still use layer-wise tickets.** The schedule control re-runs all three training conditions under global pruning, but the pipeline and ε -sweep experiments do not have global counterparts, so their tails still carry the confound. The small- ε “pruning helps robustness” result in particular might be a first-layer effect rather than a sparsity effect, and that is the most useful follow-up.
- **Global pruning has its own pathology.** It empties a later convolutional layer to 0.3% while keeping the first layer at 26.7%, so it does not avoid destroying a layer so much as choose a different one. The two schedules bracket the question rather than settling it; a sensitivity-scaled rate is the principled version neither implements.
- **Tickets come from standard training only.** All conditions evaluate masks found by standard-trained pruning, which isolates the variable of interest but measures standard-training tickets used under other conditions, a narrower claim than running separate pruning per condition.
- **One architecture, one dataset.** MNIST and LeNet-5 are a testbed, not evidence about modern networks. The pruning-helps-robustness result especially could be specific to a model with this little capacity.
- $n = 5$. Adequate for effects above roughly 0.1 points, underpowered for the several comparisons in Section 5 that failed.

8 | Conclusion

This project began with a simple question, is pruning that is free on clean MNIST also free under corruption and attack, and arrived at a two-part answer that a control experiment was needed to pin down. Under the textbook layer-wise recipe the compression is not free: at 3.75% of weights it costs seven times more corruption accuracy than clean accuracy, a gap that clean-accuracy validation cannot see. But the schedule control shows that most of this penalty is the recipe, not the sparsity, global pruning at matched sparsity erases the corruption penalty and is strictly better on both axes until roughly 1% of weights remain.

The contribution is therefore less a new method than a corrected measurement: a widely used compression recipe carries a hidden robustness cost, that cost is attributable to how weights are selected rather than how many are removed, and a better-established selection rule removes it at no price. Two secondary findings came from the same data, distillation recovers part of the pruning cost for free, and INT8 quantization buys $3.28\times$ size at the cost of latency on a model this small. Every result is reported with its error bar and its failed tests alongside, on the principle that an honest negative is worth more than an overstated positive.

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE.
2. Frankle, J., Carbin, M. (2019). *The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks*. ICLR.
3. Frankle, J., Dziugaite, G.K., Roy, D.M., Carbin, M. (2020). *Linear Mode Connectivity and the Lottery Ticket Hypothesis*. ICML.

4. Han, S., Pool, J., Tran, J., Dally, W. (2015). *Learning both Weights and Connections for Efficient Neural Networks*. NeurIPS.
5. Hinton, G., Vinyals, O., Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. NeurIPS Deep Learning Workshop.
6. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A. (2018). *Towards Deep Learning Models Resistant to Adversarial Attacks*. ICLR.
7. Mu, N., Gilmer, J. (2019). *MNIST-C: A Robustness Benchmark for Computer Vision*. ICML Workshop on Uncertainty and Robustness in Deep Learning.
8. Goodfellow, I., Shlens, J., Szegedy, C. (2015). *Explaining and Harnessing Adversarial Examples*. ICLR.